

wormhole@2新特性提案

概述

将 wormhole 扩展至一个通用的 openapi 自动生成工具，利用自定义模板特性预定义 axios、fetch、ky 等请求模板，可为用户提供这些请求工具的代码生成，为广大请求工具赋能 ai skills、编辑器内嵌文档、自定义 openapi 文档能力。

自定义模板

背景

1. 提升灵活性，实现多模板自定义切换，解决自动生成的代码 treeshaking，同时避免一个文件过多内容导致维护性差，代码冲突难解决问题。
2. 支持用户自定义模板。
3. 除了预定义 alova 模板外，额外新增 axios、fetch、ky 等模板，将 wormhole 的特性扩大使用范围

配置设计修改

代码块

```
1  import { defineConfig } from '@alova/wormhole';
2  import { rename } from '@alova/wormhole/plugin';
3  import { globals, functional } from '@alova/wormhole/template';
4
5  module.exports = defineConfig({
6    // api生成设置数组，每项代表一个自动生成的规则，包含生成的输入输出目录、规范文件地址等等
7    generator: [
8      {
9        // --- 新增或修改的配置项 start ---
10       docComment: false, // 是否生成文档注释，默认为true，设置为false可提高生成性能
11
12       responseMediaType: 'application/json', // 也可以设置为数组，表示依次指向的
           media type
```

```

13     bodyMediaType: 'application/json', // 也可以设置为数组，表示依次指向的media
    type
14
15     // 配置多个api文档时的自定义的serverName，用于在侧边栏展示的服务器名称，未设置时
    默认为当前openapi文件的info.title值，如果为空则为生成路径（例如src/api）
16     serverName: 'xxxx',
17
18     // 模板设置，指定用哪套模板生成代码，它接收一个同步或异步函数，返回一个模板路径的字
    符串（基于项目根目录），预定义了多套模板：
19     // 1. functional: 函数式模板，生成函数式调用代码，支持treeshaking，仅支持alova3
20     // 2. globals: 全局模板，现有的全局模板
21     // 3. axios: axios相关模板
22     // 4. fetch: fetch相关模板
23     // 5. ky: ky相关模板
24     template: globals({
25         // --- 模板参数 ---
26
27         /**
28          * 全局导出的api名称，可通过此名称全局范围访问自动生成的api，默认为`Apis`，配置
    了多个generator时为必填，且不可以重复
29          */
30         global: 'Apis',
31
32         /**
33          * 全局api对象挂载的宿主对象，默认为`globalThis`，在浏览器中代表`window`，
    在nodejs中代表`global`
34          */
35         globalHost: 'globalThis',
36         useImportType: true,
37     }),
38     // --- 新增或修改的配置项 end ---
39 }
40 ]
41 });

```

简易模板配置

可定义`\.alovarc`编写 openapi 文件地址即可快速设置，多个 openapi 文件地址可换行，`\#`号为单行注释

代码块

```
1 # 将会在src/api文件夹下生成，默认以alova作为模板
```

```
2 https://xxxx.com/openapi.json
3
4 # 将会在src/api2文件夹下生成，以axios作为模板
5 https://yyyy.com/openapi.json, axios
6
7 # 也可以通过设置key自定义文件夹名
8 myApi=https://zzzz.com/openapi.json // 将会在根目录myApi文件夹下生成
9 public/myApi=https://zzzz.com/openapi.json // 将会在根目录public/myApi文件夹下生成
```

模板机制设计

配置

模板可通过 `template` 配置项指定，它接收一个同步或异步函数，定义如下：

代码块

```
1 interface TemplateConfig {
2   // path: 模板路径的字符串（基于项目根目录）用于指定应加载模板的路径（相对或绝对都可以，
   // 相对是相对于process.cwd()）
3   path: './node_modules/@alova/wormhole/templates/globals',
4
5   // config: 模板参数，可根据需要自定义
6   config: {
7     // ...
8   }
9 }
10
11 function templateConfig(): TemplateConfig | Promise<TemplateConfig>;
```

区分模板的模块类型

为了将模板扩展通用性，在 `template` 指定的路径下，可分别提供 `typescript/module/common` 不同模块类型的模板，然后还是根据 `type` 字段来区分应该选择哪一套模板（`type` 字段设置参数与原先保持一致），使用 `handlebars` 模板引擎，举例：

代码块

```
1 - typescript
```

```
2 - index.ts.handlebars
3 - index.d.ts.handlebars
4 - module
5 - index.mjs.handlebars
6 - index.d.ts.handlebars
7 - common
8 - index.cjs.handlebars
9 - index.d.cts.handlebars
```

根据 type 值来选择对应的模板，type 值依然支持 `auto`。模板可以只提供部分模块类型目录（例如只有 `typescript/` 和 `module/`，没有 `common/`），此时若用户请求了不存在的模块类型，工具将抛出明确的错误信息，列出该模板实际支持的类型，例如：

代码块

```
1 Template "ky" does not support module type "commonjs". Supported types:
  typescript, module
```

如果自定义模板没有进行区分模块类型，则将 template 路径下的都当作目标模板来生成，举例：

代码块

```
1 - folder-A
2 - #index.ts.handlebars
3 - index.d.ts.handlebars
4 - folder-B
5 - index.ts.handlebars
6 - index.d.ts.handlebars
```

此时以上两个文件夹都当作目标模板来生成。

值得注意的是，生成时，生成代码的文件结构保持与模板文件结构一致。

支持 partial 引入

模板根目录下 partials 文件夹内的模块自动被注册为 partial，可通过文件名进行引入，例如：

代码块

```
1 - partials
2   - partial1.handlebars
3 - folder-A
4   - index.ts.handlebars
```

在 `folder-A/index.ts.handlebars` 中可通过`\{\> partial1\}`引用。

文件生成规则

在之前的版本中，固定生成 `index.ts`、`globals.d.ts`、`apiDefinitions` 等文件，在本次版本中需要升级为可自定义生成文件：

1. 按 tag 名区分文件：文件名可指定为 `{tag}`（例如 `{tag}.js`），生成文件时将遍历 openapi 文件中的 tag，然后生成对应个数的文件，将 `{tag}` 替换为对应的实际 tag，不包含中括号。这些 tag 会被预处理成不包含中文的文本（在当前版本中已实现）。

不可覆盖的文件

可在文件名称前添加一个 `#`，表示文件不覆盖，与当前的 `index.ts` 生成逻辑相同。例如 `#index.ts` 文件生成时会检查是否已存在，不存在才生成，在生成后，对应的文件名需要去掉 `#` 号。

例如，上面模板中的 `folder-A/#index.ts`，在生成时将会检查 `folder-A/index.ts` 是否存在，存在则不生成，不存在则生成，没有 `#` 号则在每次生成都覆盖处理。

模板参数

每个模板在生成时，都将会接收到以下模板参数，这是一个标准化的模板渲染参数设计：

代码块

```
1 /**
2  * 引入描述符 — 内部自动生成，每个 API 各不相同
```

```

3      *
4      * 用于在 AI Skill 文档中根据模板类型自适应生成 import/require 语句:
5      * - typescript/module (ESM): import { name } from 'source'
6      * - commonjs:                const { name } = require('source')
7      *
8      * 不需要用户配置, 由模板预设自动推导:
9      * - 按需引入模板 (functional/axios/fetch/ky) : names=[api.name],
      source='./<tag>'
10     * - 全局挂载模板 (alovaGlobals) : needed=false ("无需引入")
11     */
12     interface ImportDescriptor {
13         /** 是否需要引入语句。false 表示"无需引入" (如 globals 全局挂载模式) */
14         needed: boolean
15         /** 需要引入的导出名称列表 */
16         names: string[]
17         /** 引入源路径 */
18         source: string
19         /** 是否为默认导出引入 (import X from ... 而非 import { X } from ...) */
20         isDefault?: boolean
21     }
22
23     interface Api {
24         tag: string
25         method: string
26         summary: string
27         path: string
28         pathParameters: string
29         queryParameters: string
30         pathParametersComment?: string
31         queryParametersComment?: string
32         responseComment?: string
33         requestComment?: string
34         name: string
35         responseName: string
36         requestName?: string
37         defaultValue?: string
38         pathKey: string
39         /** 引入描述符, 内部自动生成, AI Skill 文档中用于生成自适应的 import/require 语句 */
40         importDescriptor?: ImportDescriptor
41     }
42
43     interface TemplateData {
44         title: OpenAPIDocument['info']['title']
45         openapi: OpenAPIDocument['openapi']
46         version: OpenAPIDocument['info']['version']
47         description: OpenAPIDocument['info']['description']
48         contact: OpenAPIDocument['info']['contact']

```

```

49  /** Framework tag: vue | react | svelte | solid-js | nuxt */
50  framework?: string
51  defaultKey?: boolean
52  baseUrl: string
53  /** Schema/Component definitions */
54  components: string[]
55  /** All apis array */
56  apis: Api[]
57  /** Apis grouped by tag */
58  taggedApis: ApiDoc[]
59  type: 'typescript' | 'module' | 'commonjs'
60  /** Config passed from template configuration */
61  config: Record<string, any>
62  }

```

缓存文件

缓存文件内容重构

alova 缓存文件就是用于渲染侧边栏 api 文档树，快速搜索 api 的数据，因为可以自定义模板，其内部的接口调用函数的名字并不是固定为 `Apis.xxx.yyy` 了，所以需要标准化缓存数据的格式来记录每个接口调用函数的名字，这样在原来的 vscode 插件中还可以快速搜索，目前的设计如下：

代码块

```

1  [
2    {
3      "path": "src/api",
4      "serverName": "server web", // 配置文件有设置才有值
5      "apis": Api[], // 伪代码, Api为对应的ts类型
6    },
7    {
8      "path": "src/api2",
9      "serverName": "server mobile",
10     "apis": Api[],
11   },
12   // ...
13 ]

```

缓存文件路径修改

原先的缓存文件放在 `node_modules` 中会导致无法同步提交，导致团队多个成员的 api 文档和快速搜索内容不一致，因此缓存文件的路径从原先的 `node_modules/.alova` 修改到根目录下的 `.alova-cache.json`。

预定义模板设计

内置 `alova`、`alovaGlobals`、`axios`、`fetch`、`ky` 五套模板。

functional

通过预定义函数 `functional` 设置

代码块

```
1 import { alova } from '@alova/wormhole/template';
2
3 export default defineConfig({
4   generator: [
5     // ...
6     template: alova()
7   ]
8 })
```

模板设计

采用**按需引入**模式，每个 API 作为独立函数从对应 tag 文件导出，支持 tree-shaking。

模板目录结构

代码块

```
1 alova-functional/typescript/
2 |—— index.ts.handlebars      # 入口：创建 alovaInstance + 重导出所有 tag 模块
3 |—— {tag}.ts.handlebars      # 按 tag 遍历生成 API 函数文件
4 |—— types.ts.handlebars      # 类型声明文件
```


> **注意：**该模板不再包含 `createApis.ts.handlebars`，因为按需引入模式下不需要构建全局 `Apis` 对象。每个 API 函数直接在 `{tag}.ts` 中定义并导出。

index.ts.handlebars

代码块

```
1  {{{commentText}}}
2  import { createAlova } from 'alova'; import { fetchAdapter } from
  'alova/fetch';
3  {{#if (eq framework "vue")}}
4    import { vueHook } from 'alova/vue';
5  {{else if (eq framework "react")}}
6    import { reactHook } from 'alova/react';
7  {{/if}}
8
9  export const alovaInstance = createAlova({ baseUrl: "{{{baseUrl}}}",
10  {{#if (eq framework "vue")}}
11    statesHook: vueHook,
12  {{else if (eq framework "react")}}
13    statesHook: reactHook,
14  {{/if}}
15  requestAdapter: fetchAdapter(), beforeRequest: method => { }, responded: res =>
16  { return res.json(); } });
17
18  {{#each taggedApis}}
19    export * from './{{{tag}}}';
20  {{/each}}
```

{tag}.ts.handlebars

代码块

```
1  {{{commentText}}}
2  import type { Method, AlovaMethodCreateConfig, AlovaGenerics } from 'alova';
3  import { Method as MethodClass } from 'alova';
4  import { alovaInstance } from './index';
5
6  {{#each taggedApis}}
7  {{#apis}}
8    /**
9     * {{{summary}}}
10    * @method {{{method}}}
```

```

11  * @path {{{path}}}
12  */
13  export function {{{name}}}(
14    config: AlovaMethodCreateConfig<any, any, any, any>{{#or pathParameters
    queryParameters requestName }} & {
15      {{#if pathParameters}}
16      pathParams: {{{pathParameters}}};
17      {{/if}}
18      {{#if queryParameters}}
19      params: {{{queryParameters}}};
20      {{/if}}
21      {{#if requestName}}
22      data: {{{requestName}}};
23      {{/if}}
24    }{{/or}} = {}
25  ): Method<any, any, any, any, any> {
26    {{#if pathParameters}}
27    const pathParams = config.pathParams || {};
28    const resolvedPath = '{{{path}}}'.replace(/\{(\w+)\}/g, (_, key) =>
    pathParams[key]);
29    {{else}}
30    const resolvedPath = '{{{path}}}';
31    {{/if}}
32
33    return new MethodClass(
34      '{{method}}',
35      alovaInstance,
36      resolvedPath,
37      config
38    );
39  }
40
41  {{/apis}}
42  {{/each}}

```

修正说明（相对于当前代码）：

1. ~~import type { alovaInstance }`~~ → `import { alovaInstance }`：`alovaInstance` 是运行时值，被传入 `new MethodClass(..., alovaInstance, ...)` 构造器，`import type` 会在编译后被擦除导致运行时报错

2. 移除 `createApis.ts.handlebars`： `createApis` / `createApiMethod` 构建全局 `Apis` 对象，与按需引入模式设计矛盾

3. `types.ts.handlebars` 中 ~~`{{#schemas}}`~~ → `{{#each components}}`：修正与数据字段名不匹配的问题

调用示例

代码块

```
1 // 按需引入, 支持 tree-shaking
2 import { getUser } from './api/user';
3 import { createArticle } from './api/content';
4
5 const user = await getUser({ params: { id: 1 } });
```

importDescriptor 自动推导

此模板每个 API 自动生成的 `importDescriptor`：

代码块

```
1 {
2   "needed": true,
3   "names": ["getUser"],
4   "source": "./user"
5 }
```

对应的自适应引入语句（由渲染引擎根据 `type` 字段生成）：

- `typescript/module`： `import { getUser } from './user'`
- `commonjs`： `const { getUser } = require('./user')`

alovaGlobals

通过预定义函数 `alovaGlobals` 设置

代码块

```
1 import { alovaGlobals } from '@alova/wormhole/template';
2
3 const templateAlovaGlobalsConfig = { ... };
4 export default defineConfig({
```

```

5     generator: [
6         // ...
7         template: alovaGlobals(templateAlovaGlobalsConfig )
8     ]
9 })

```

支持参数与原先的参数相同：

代码块

```

1  interface TemplateAlovaGlobalsConfig {
2      /**
3       * 全局导出的api名称，可通过此名称全局范围访问自动生成的api，默认为`Apis`，配置了多个
4       * generator时为必填，且不可以重复
5       */
6       global?: string,
7       /**
8       * 全局api对象挂载的宿主对象，默认为`globalThis`，在浏览器中代表`window`，在
9       * nodejs中代表`global`
10      */
11      globalHost?: string,
12      /**
13       * 去掉此参数，只能生成alova@3版本，不再支持v2，之前的v2版本模板去掉
14       */
15      ~~// version: 'v3',~~
16      // 使用import type方式导入，默认false
17      useImportType?: boolean
18  }

```

模板设计：暂无

axios

通过预定义函数 axios 设置

代码块

```

1  import { axios } from '@alova/wormhole/template';
2

```

```

3   export default defineConfig({
4     generator: [
5       // ...
6       template: axios()
7     ]
8   })

```

模板设计

采用与 **functional** 一致的**按需引入**模式，每个 API 作为独立函数从对应 tag 文件导出。

模板目录结构

代码块

```

1   axios/typescript/
2   |—— index.ts.handlebars      # 入口：创建 axiosInstance + 重导出所有 tag 模块
3   |—— {tag}.ts.handlebars      # 按 tag 遍历生成 API 函数文件
4   |—— types.ts.handlebars      # 类型声明文件

```

`index.ts.handlebars`

代码块

```

1   {{{commentText}}}
2   import axios from 'axios'; export const axiosInstance = axios.create({ baseUrl:
3   '{{{baseUrl}}}', });
4
5   {{#each taggedApis}}
6     export * from './{{{tag}}}';
7   {{/each}}

```

`{tag}.ts.handlebars`

代码块

```
1  {{{commentText}}}
2  import type { AxiosRequestConfig } from 'axios'; import { axiosInstance } from
3  './index';
4
5  {{#each taggedApis}}
6    {{#apis}}
7      /** *
8      {{summary}}
9      * @method
10     {{method}}
11     * @path
12     {{{path}}}
13     */ export const
14     {{{name}}}
15     = (config: AxiosRequestConfig{{#or
16       pathParameters queryParameters requestName
17     }}
18       & {
19         {{#if pathParameters}}
20           pathParams:
21             {{{pathParameters}}};
22         {{/if}}
23         {{#if queryParameters}}
24           params:
25             {{{queryParameters}}};
26         {{/if}}
27         {{#if requestName}}
28           data:
29             {{{requestName}}};
30         {{/if}}
31       }{{/or}}
32     = {}) => {
33       {{#if pathParameters}}
34         const { pathParams, ...restConfig } = config; const resolvedPath =
35         '{{{path}}}'.replace(/\{(\w+)\}/g,
36         (_, key) => pathParams[key]);
37       {{else}}
38         const restConfig = config; const resolvedPath = '{{{path}}}'
39       {{/if}}
40
41       return axiosInstance({ method: '{{method}}', url: resolvedPath,
42         ...restConfig, }); };
```

```
43   {{/apis}}
44   {{/each}}
```

调用示例

代码块

```
1  // 按需引入, 支持 tree-shaking
2  import { getUser } from "./api/user";
3
4  const user = await getUser({ params: { id: 1 } });
```

importDescriptor 自动推导

与 functional 一致, 每个 API 自动生成:

代码块

```
1  {
2    "needed": true,
3    "names": ["getUser"],
4    "source": "./user"
5  }
```

fetch

通过预定义函数 fetch 设置

代码块

```
1  import { fetch } from '@alova/wormhole/template';
2
```

```
3   export default defineConfig({
4     generator: [
5       // ...
6       template: fetch()
7     ]
8   })
```

模板设计

采用与 **functional** 一致的**按需引入**模式。

模板目录结构

代码块

```
1   fetch/typescript/
2   |—— index.ts.handlebars
3   |—— {tag}.ts.handlebars
4   |—— types.ts.handlebars
```

`index.ts.handlebars`

代码块

```
1   {{{commentText}}}
2   export const BASE_URL = '{{{baseUrl}}}'
3
4   {{#each taggedApis}}
5     export * from './{{{tag}}}'
6   {{/each}}
```

`{tag}.ts.handlebars`

代码块

```
1  {{{commentText}}}
2  import { BASE_URL } from './index';
3
4  {{#each taggedApis}}
5      {{#apis}}
6          /** *
7              {{summary}}
8              * @method
9              {{method}}
10             * @path
11             {{{path}}}
12             */ export const
13             {{{name}}}
14             = async (config: RequestInit{{#or
15                 pathParameters queryParameters requestName
16             }}
17                 & {
18                     {{#if pathParameters}}
19                         pathParams:
20                             {{{pathParameters}}};
21                     {{/if}}
22                     {{#if queryParameters}}
23                         params:
24                             {{{queryParameters}}};
25                     {{/if}}
26                     {{#if requestName}}
27                         data:
28                             {{{requestName}}};
29                     {{/if}}
30                 }{{/or}}
31             = {}) => {
32                 {{#if pathParameters}}
33                     const { pathParams, ...restConfig } = config; let resolvedPath =
34                     '{{{path}}}'.replace(/\{(\w+)\}/g,
35                     (_, key) => pathParams[key]);
36                 {{else}}
37                     const restConfig = config; let resolvedPath = '{{{path}}}'
38                 {{/if}}
39                 {{#if queryParameters}}
40                     if (restConfig.params) { const searchParams = new
41                     URLSearchParams(restConfig.params); resolvedPath += '?' +
42                     searchParams.toString(); }
43                 {{/if}}
44             }
```

```
45     const response = await fetch(BASE_URL + resolvedPath, { method:
    '{{method}}',
46     ...restConfig, }); if (!response.ok) { throw new Error(`HTTP error! status:
    ${response.status}`); } return response.json(); };
48
49     {{/apis}}
50     {{/each}}
```

调用示例

代码块

```
1  import { getUser } from "./api/user";
2
3  const user = await getUser({ params: { id: 1 } });
```

importDescriptor 自动推导

与 functional 一致:

代码块

```
1  {
2    "needed": true,
3    "names": ["getUser"],
4    "source": "./user"
5  }
```

ky

通过预定义函数 ky 设置

```

1 代码块
import { ky } from '@alova/wormhole/template';
2
3  export default defineConfig({
4     generator: [
5         // ...
6         template: ky()
7     ]
8 })

```

模板设计

采用与 **functional** 一致的**按需引入**模式。

模板目录结构

代码块

```

1  ky/typescript/
2  |—— index.ts.handlebars
3  |—— {tag}.ts.handlebars
4  |—— types.ts.handlebars

```

`index.ts.handlebars`

代码块

```

1  {{{commentText}}}
2  import ky from 'ky'; export const kyInstance = ky.create({ prefixUrl:
   {{{baseUrl}}}'
3  });
4
5  {{#each taggedApis}}
6     export * from './{{{tag}}}';
7  {{/each}}

```

`{tag}.ts.handlebars`

代码块

```
1  {{{commentText}}}
2  import type { Options } from 'ky'; import { kyInstance } from './index';
3
4  {{#each taggedApis}}
5    {{#apis}}
6      /** *
7        {{summary}}
8        * @method
9        {{method}}
10       * @path
11       {{{path}}}
12      */ export const
13      {{{name}}}
14      = (config: Options{{#or pathParameters queryParameters requestName}}
15        & {
16          {{#if pathParameters}}
17            pathParams:
18              {{{pathParameters}}};
19          {{/if}}
20          {{#if queryParameters}}
21            searchParams:
22              {{{queryParameters}}};
23          {{/if}}
24          {{#if requestName}}
25            json:
26              {{{requestName}}};
27          {{/if}}
28          {{/or}}
29        = {}) => {
30          {{#if pathParameters}}
31            const { pathParams, ...restConfig } = config; const resolvedPath =
32              '{{{path}}}'.replace(/\{(\w+)\}/g,
33                (_, key) => pathParams[key]);
34          {{else}}
35            const restConfig = config; const resolvedPath = '{{{path}}}'
36          {{/if}}
37
38          return kyInstance(resolvedPath, { method: '{{method}}', ...restConfig,
39            }).json(); };
```

```
40     {{/apis}}
41     {{/each}}
```

调用示例

代码块

```
1  import { getUser } from "./api/user";
2
3  const user = await getUser({ searchParams: { id: 1 } });
```

importDescriptor 自动推导

与 functional 一致：

代码块

```
1  {
2    "needed": true,
3    "names": ["getUser"],
4    "source": "./user"
5  }
```

其他修改

1. 去除配置文件项的 fileNameCase 配置项及相关功能
2. 去除 `autoUpdate` 配置项（`Config` 根配置项）及相关功能，包括 `getAutoUpdateConfig` 导出函数；自动更新逻辑后续由 VSCode 扩展自行管理
3. CLI 命令行参数重新设计

设计原则

- `-c` 在大多数 CLI 工具中惯例为 `--config`（指定配置文件），原先用作 `--cwd` 容易产生误解，改为 `-p, --project` 更清晰
- `gen` 命令默认以 workspace 模式运行（扫描所有子项目），当用户通过 `-p` 指定具体项目目录时则只对该项目生成
- 去掉 `-w, --workspace` 参数，因为 workspace 模式即为默认行为，无需额外 flag

命令设计

代码块

```
1  # init 命令: 初始化配置文件
2  alova init [-t, --type <type>] [-T, --template <template>] [-p, --project <path>]
3
4  # gen 命令: 生成 API 代码
5  alova gen [-f, --force] [-p, --project <path>]
```

命令	参数	说明
init	-t, --type <type>	配置文件类型，限定 typescript / ts / commonjs / module
init	-T, --template <template>	初始化使用的模板预设，限定 alova / functional / axios / fetch / ky，默认为 alova
init	-p, --project <path>	指定项目目录（alova.config 所在目录）
gen	-f, --force	强制重新生成（忽略缓存）
gen	-p, --project <path>	指定项目目录，传入后仅对该项目生成；未传入时以 workspace 模式扫描所有子项目

行为逻辑

代码块

```
1  alova gen
2    → 默认 workspace 模式：调用 resolveWorkspaces() 扫描所有子项目，逐一生成
3
4  alova gen -p ./packages/app
5    → 单项目模式：仅对指定目录读取配置并生成
```

其他改进

- `--type` 参数使用 commander 的 `.choices()` 在 CLI 层直接校验合法值，拼写错误时立即报错
- 生成过程加入 try/catch 错误处理，异常时正确终止 spinner 并显示错误信息
- 非 workspace 模式的成功/失败提示不再显示 `undefined`，改为显示实际项目路径或 `.`

插件系统改进

Hook 命名规范

将所有 `afterXxx` 形式的 hook 名称改为过去式 `xxxed`，更符合事件命名语义：

旧名称	新名称
<code>afterOpenapiParse</code>	<code>openapiParsed</code>
<code>afterCodeGenerate</code>	<code>codeGenerated</code>

`beforeOpenapiParse` 和 `beforeCodeGenerate` 保持不变。

Hook 参数对象化

所有 hook 参数统一改为对象形式，以便后续扩展而不破坏签名：

代码块

```
1  /**
2   * Function injected into plugin hooks for reporting plugin-scoped progress.
3   */
4  export type ReportProgress = (progress: number, message?: string) => void
5
6  interface ApiPlugin {
7    name?: string
8
9    config?: (params: {
10      config: GeneratorConfig
11      projectPath: string
12      reportProgress: ReportProgress
13    }) => MaybePromise<GeneratorConfig | undefined | null | void>
14
15    beforeOpenapiParse?: (params: {
16      config: Readonly<GeneratorConfig>
17      projectPath: string
18      reportProgress: ReportProgress
19    }) => void
20
21    openapiParsed?: (params: {
22      config: Readonly<GeneratorConfig>
23      document: OpenAPIDocument
24      projectPath: string
25      reportProgress: ReportProgress
26    }) => MaybePromise<OpenAPIDocument | undefined | null | void>
27
28    beforeCodeGenerate?: (params: {
29      config: Readonly<GeneratorConfig>
30      data: TemplateData
31      projectPath: string
32      reportProgress: ReportProgress
33    }) => MaybePromise<string | undefined | null | void>
34
35    codeGenerated?: (params: {
36      config: Readonly<GeneratorConfig>
```



```
37     data: TemplateData
38     files: Record<string, string>
39     projectPath: string
40     error?: Error
41     reportProgress: ReportProgress
42   }) => MaybePromise<void>
43 }
```

Config 与 projectPath 参数注入

每个 hook 都会接收 `config` 和 `projectPath` 参数：

- `config` hook：接收可修改的 config 对象
- `beforeOpenapiParse` 及之后的 hook：接收 `Object.freeze()` 冻结的 config，防止误修改
- `projectPath`：当前项目路径，所有 hook 均可获取

codeGenerated 文件修改能力

`codeGenerated` hook 新增 `files` 参数（`Record<string, string>`），key 为文件名，value 为文件内容。插件可在此 hook 中直接修改 `files` 对象的内容，修改后的结果会被写入磁盘。

生成流程调整为：

1. 模板渲染生成文件内容到内存中的 `files` 对象
2. 调用 `codeGenerated` hook，插件可修改 `files`
3. 将最终的 `files` 内容写入磁盘

importType 插件改进

`importType` 插件的函数签名更新为：

```

1 代码块 import { importType } from '@alova/wormhole/plugin';
2
3  function importType(
4     imports: Record<string, string[]>,
5     options?: { files?: string[] }
6  ): ApiPlugin;

```

参数说明

- `imports`：一个对象，key 为模块路径（可附加 `|type` 标记），value 为需要引入的类型名称数组。
 - 当 key 包含 `|type` 后缀时（如 `'module-name|type'`），生成 `import type { ... }` 语句
 - 否则生成普通的 `import { ... }` 语句
- `options.files`：可选，指定需要插入 import 语句的目标文件名匹配列表，默认为 `['globals.d']`

行为逻辑

1. **config hook**：将 `imports` 中所有类型名称添加到 `config.externalTypes`，使这些类型在生成时被跳过（直接引用而不重新生成）。
2. **codeGenerated hook**：在匹配 `options.files` 的生成文件顶部（紧跟块注释之后）插入对应的 import 语句。

使用示例

代码块

```

1  import { defineConfig } from '@alova/wormhole';
2  import { importType } from '@alova/wormhole/plugin';
3
4  export default defineConfig({
5    generator: [
6      {
7        plugins: [
8          importType(
9            {

```

```
10         'my-types|type': ['Pagination', 'BaseResponse'],
11         '@/shared': ['File', 'FormData'],
12     },
13     { files: ['globals.d', 'types'] }
14 ),
15 ],
16 }
17 ]
18 });
```

以上配置将生成：

代码块

```
1 import type { Pagination, BaseResponse } from "my-types";
2 import { File, FormData } from "@/shared";
```

并将 `Pagination`、`BaseResponse`、`File`、`FormData` 排除在自动生成之外。

AI skill

背景

vibe coding 趋势下，ai 生成接口调用代码时不知道有哪些接口调用，也不知道参数信息，因此提供生成 ai 友好的特定 apis 的 skill

ai 文档生成插件设计

新增 aiDoc 插件，此插件将会先生成对应的 api\skill，再使用 `skills add localpath` 的方式安装此 skill，依赖 skills 库

代码块

```
1  const aiPlugin = aiDoc({
2    template: './custom-ai-skills'
3  })
4
5  defineConfig({
6    generator: [
7      {
8        plugins: [aiPlugin]
9      }
10   ]
11  })
```

自定义 skill 模板

handlebars 模板，使用方法与模板参数同上

alova 文档内容设计

SKILL*.md

代码块

```
1  ---
2  name: generated-APIs
3  description: ...
4  ---
5
6  # Skill for generated APIs
7
8  ### 执行任务
9
10  根据当前的接口需求阅读对应接口的详细文档，以获知接口的详细参数信息和调用方式
11
12  ### 接口目录(server1)
13
14  - [获取用户信息](src/api/aidocs/user/info.md)
```

- 15 - [用户登录]([src/api/aidocs/user/login.md](#))
- 16 - [管理员权限配置]([src/api/aidocs/admin/permissions.md](#))
- 17 - [系统操作日志审计]([src/api/aidocs/admin/audit.md](#))
- 18 - [全局通知管理]([src/api/aidocs/admin/notifications.md](#))
- 19 - [文章内容管理 (CRUD)]([src/api/aidocs/content/articles.md](#))
- 20 - [用户评论与回复]([src/api/aidocs/content/comments.md](#))
- 21 - [文件上传与资源管理]([src/api/aidocs/content/uploads.md](#))

references/xxx*.md

以 **functional 模板**（按需引入）为例，每个接口生成一个独立文档：

代码块

```
1  ### 重要：在生成代码前强制执行（不得违反）！！
2
3  在执行代码生成任务前复述以下的使用约定内容，并在生成时确保遵守使用约定
4
5  ## 使用约定
6
7  1. 此文档所标记的接口均使用alova作为请求客户端，需遵循alova的使用规范
8  2. 需要按需引入对应的API函数，如：`import { selectWarnSecurity } from
   './api/SwapAccountManageController'`
9  **
10
11 #### 接口**
12
13 管理员权限配置
14
15 #### Path Parameters**
16
17 根据实际需要在`pathParams`中传入参数
18
19 ```typescript
20 interface PathParameters {
21   // 账户编号
22   accountCode: string;
23 }
```

Request Body

根据实际需要在 `data` 中传入请求体

代码块

```
1 interface RequestBody {
2     // 操作人租户ID, 无填0
3     operatorTenantId?: string;
4     // 页码
5     currPage?: number;
6     // 每页条数
7     pageSize?: number;
8     // 账户编号
9     accountCode?: string;
10    // 账单类型
11    billType?: string[];
12    // 创建时间开始时间
13    startTime?: string;
14    // 创建时间结束时间
15    endTime?: string;
16    // 收支类型: 收入、支出
17    incomeOrExpenses?: string;
18    // 账单编号
19    billCode?: string;
20 }
```

Response

请根据实际需要使用响应数据

代码块

```
1 interface Response {
2     // 账户编号
3     accountCode?: string;
4     // 余额提醒单价
5     securityDepositUnitPrice?: number;
6     // 余额提醒上限
```

```
7   securityDepositMax?: number;
8 }
```

调用示例

以下调用将返回一个 alova 的 Method 实例。

代码块

```
1 import { selectWarnSecurity } from "../api/SwapAccountManageController";
2
3 selectWarnSecurity({
4   pathParams: {
5     accountCode: "",
6   },
7   data: {},
8 });
```

代码块

```
1
2 ---
3
4 ## importDescriptor — 引入路径自适应机制
5
6 ### 设计原则
7
8 `importDescriptor` 是 `Api` 上的一个系统内部自动计算的字段，**不需要用户配置**。它根据
  模板类型自动推导：
9
10 | 模板类型 | `importDescriptor.needed` | 说明 |
11 |-----|-----|-----|
12 | **functional** / **axios** / **fetch** / **ky** | `true` | 按需引入。`names=
  [api.name]`，`source='../<tag>` |
13 | **alovaGlobals** | `false` | 全局挂载在 `globalThis` 上，无需引入 |
14 | 自定义模板（含 `{tag}` 文件） | `true` | 自动检测到 `{tag}` 模板即视为按需引入模式 |
15 | 自定义模板（无 `{tag}` 文件） | `false` | 默认视为无需引入 |
16
17 ### importDescriptor 的类型定义
```

```
18
19  ```typescript
20  interface ImportDescriptor {
21    /** 是否需要引入语句。false → "无需引入" */
22    needed: boolean
23    /** 需引入的导出名称列表 */
24    names: string[]
25    /** 引入源路径 */
26    source: string
27    /** 是否为默认导出引入 */
28    isDefault?: boolean
29  }
```

根据 type 自适应渲染

在 AI Skill 文档渲染时，根据 `TemplateData.type` 字段自适应生成引入语句：

type 值	ESM (<code>import</code>)	CommonJS (<code>require</code>)
typescript	<code>import { getUser } from './user'</code>	—
module	<code>import { getUser } from './user'</code>	—
commonjs	—	<code>const { getUser } = require('./user')</code>

渲染逻辑（在 skill 模板 handlebars helper 中实现）：

代码块

```
1  function renderImportStatement(descriptor, type) {
2    if (!descriptor || !descriptor.needed) return "无需引入";
3
4    const names = descriptor.names.join(", ");
5    const source = descriptor.source;
```



```

6
7   if (type === "commonjs") {
8       return descriptor.isDefault
9       ? `const ${names} = require('${source}');`
10      : `const { ${names} } = require('${source}');`;
11   }
12   // typescript / module
13   return descriptor.isDefault
14   ? `import ${names} from '${source}';`
15   : `import { ${names} } from '${source}';`;
16 }

```

路径解析优先级

`importDescriptor.source` 中的路径按以下优先级解析：

1. **tsconfig/jsconfig paths 别名匹配** — 读取项目根目录的 `tsconfig.json` 或 `jsconfig.json` 中的 `compilerOptions.paths`，将源码路径映射为别名路径
2. **回退到相对路径** — 如果没有匹配的 `paths` 别名，使用相对于输出目录的路径（即 `source` 原始值）

代码块

```

1  示例：
2      source = './user'
3      tsconfig.paths = { "@api/*": ["/src/api/*"] }
4      输出目录 = /project/src/api
5      → 解析结果：@api/user
6
7      source = './user'
8      无 tsconfig.paths
9      输出目录 = /project/src/api
10     → 解析结果：./user

```

生成进度上报

背景

OpenAPI 文档体量较大或网络不稳定时，整个生成流程可能耗时数十秒。原版 CLI 仅显示一个不变的 spinner，使用方无法判断当前处于哪个阶段、是否卡住，也无法在 IDE / Web 集成中实现细粒度的进度展示。本特性新增三层可观察性：

1. `generate(config, options)` 暴露 `onProgress` 回调和 `progressInterval` 节流参数。
2. 每个插件生命周期钩子接收一个 `reportProgress` 回调，回报的事件以插件 `name` 标识，便于上层按来源分组。
3. CLI `gen` 命令默认 500ms 刷新 spinner，按来源（`core` 与各插件）多行展示百分比与文案。

接口设计

`GenerateApiOptions` 扩展

代码块

```
1  interface GenerateProgress {
2      /** 'core' 表示框架核心生成阶段；其他值为插件 name */
3      source: string;
4      /** 0-100 之间的整数百分比，越界时被框架自动 clamp */
5      progress: number;
6      /** 可选的人类可读消息 */
7      message?: string;
8  }
9
10 interface GenerateApiOptions {
11     force?: boolean;
12     projectPath?: string;
13     /**
14      * 进度回调。框架按 progressInterval 节流后向其推送当前所有 source 的最新进度。
15      * 回调接收 `Record<source, GenerateProgress>` 快照对象。
16      */
17     onProgress?: (snapshot: Record<string, GenerateProgress>) => void;
18     /**
19      * 推送间隔（毫秒），默认 `500`。
20      * `<= 0` 时关闭节流，每次 update 即触发回调。
21      */
22 }
```

```
22   progressInterval?: number;
23 }
```

插件钩子注入 reportProgress

5 个生命周期钩子（`config` / `beforeOpenapiParse` / `openapiParsed` / `beforeCodeGenerate` / `codeGenerated`）的参数对象统一新增 `reportProgress` 字段：

代码块

```
1  type ReportProgress = (progress: number, message?: string) => void;
```

框架在调用钩子时为每个插件实例绑定一个独立的 `reportProgress`，其内部以该插件的 `name` 作为 source 标识。插件无需关心如何向上传递事件——直接调用即可。如果插件未声明 `name`，则统一以 `'plugin'` 作为 source（多个匿名插件之间会发生覆盖，提示用户填写 name）。

代码块

```
1  import { defineConfig } from "@alova/wormhole";
2
3  export default defineConfig({
4    generator: [
5      {
6        plugins: [
7          {
8            name: "remoteSchema",
9            async openapiParsed({ document, reportProgress }) {
10              reportProgress(10, "fetching schema patches");
11              await fetchPatches();
12              reportProgress(60, "merging");
13              await merge(document);
14              reportProgress(100, "done");
15            },
16          },
17        ],
18      },
19    ],
```

```
20    });
```

核心阶段进度锚点

`GeneratorHelper.generate` 的 `core` source 在以下检查点上报进度：

百分比	阶段
5	starting
10	beforeOpenapiParse
20	parsing openapi document
35	openapi parsed
45	openapiParsed (after plugin chain)
55	template configuration loaded
65	template data parsed
70	beforeCodeGenerate
80	processing templates
90	template generation completed
95	writing files
100	completed

跳过路径（`No OpenAPI document found` / `Template data unchanged`）和异常路径同样会上报 100 终态，消息以 `skipped:` 或 `failed:` 前缀，保证 CLI 终态可识别。

CLI 展示

alova gen 默认接入 onProgress，将 spinner 文本替换为多行渲染：

代码块

```

1 Generating `./packages/app`...
2 [core] ██████████░░░░░░░░░░ 60% template data parsed
3 [aiDoc] ███████░░░░░░░░░░░░ 20% rendering references
```

排序规则：`core` 行始终在最上，插件按 `name` 字典序排列。渲染逻辑被抽取为独立的 `progressRenderer.ts`（`formatProgressBar` / `sortProgressEntries` / `renderProgressSnapshot` / `createProgressRenderer`），既可复用于其他 CLI/IDE 集成，也便于单元测试。

兼容性

- `GenerateApiOptions` 新增字段全部可选，旧调用方零改动。
- 钩子参数对象新增 `reportProgress` 属性；原有插件不调用即可，行为不变。
- 不引入新的运行时依赖（CLI 仍使用 `ora`）。